УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 21

СОЗДАНИЕ И ПРАВКА СТРУКТУРЫ ПРОЕКТА. ЗАПУСК СОЗДАННОГО ПРИЛОЖЕНИЯ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание и правка структуры проекта. Запуск созданного приложения

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Работа со статическими файлами

Центральным моментом любого веб-приложения является обработка запроса, который отправляет пользователь. В Django за обработку запроса отвечают представления или views. По сути представления представляют функции обработки, которые принимают данные запроса в виде объекта HttpRequest из пакета django.http и генерируют некоторый результат, который затем отправляется пользователю.

Обычно в веб-приложениях используются различные статические файлы, такие как изображения, файлы css (для стилей), javascript и т.д. Давайте рассмотрим, как эти файлы могут быть использованы.

При создании проекта Django уже предоставляет базовые настройки для работы со статическими файлами. Например, в файле settings.py определена переменная STATIC_URL, которая содержит адрес каталога со статическими ресурсами (рисунок 1).

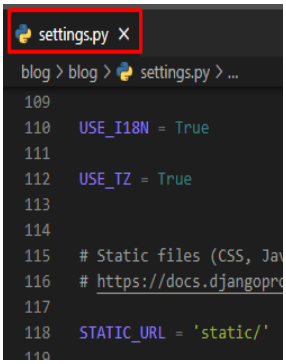


Рисунок 1 - Значение по умолчанию

А среди установленных приложений в переменной INSTALLED_APPS

указано приложение django.contrib.staticfiles (рисунок 2).

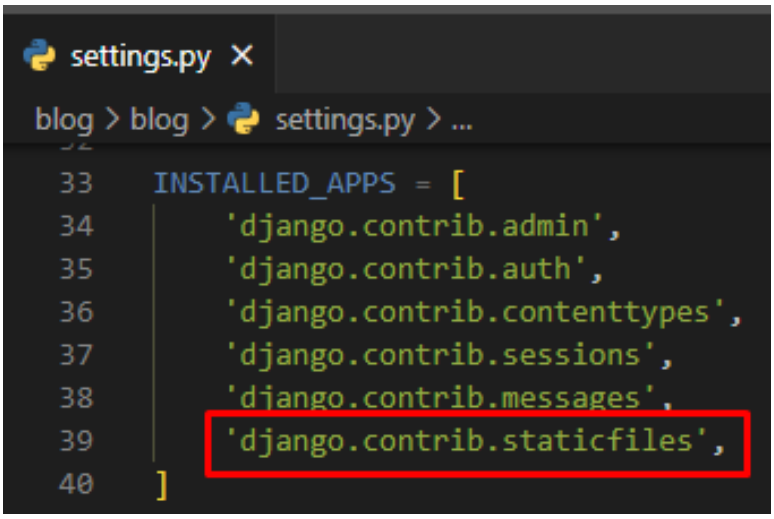


Рисунок 2 - django.contrib.statcfiles в INSTALLED_APPS

Переменная STATIC_URL установлена на "static/", что означает, что мы можем просто создать каталог с названием "static" внутри папки нашего приложения и добавить туда нужные статические ресурсы. Однако, при необходимости, мы всегда можем изменить расположение каталога статических файлов через эту настройку.

Итак, мы добавляем новый каталог "static" внутри нашего приложения. Чтобы не смешивать все статические файлы в одной папке, мы решаем организовать разные каталоги для различных видов файлов. Например, мы создаем каталог "images" внутри "static" для изображений, и отдельный каталог "css" для файлов стилей. Точно так же мы можем создать папки и для других видов файлов.

Давайте создадим следующую иерархию файлов и поместим фотографию ночного неба в папку images (рисунок 3):

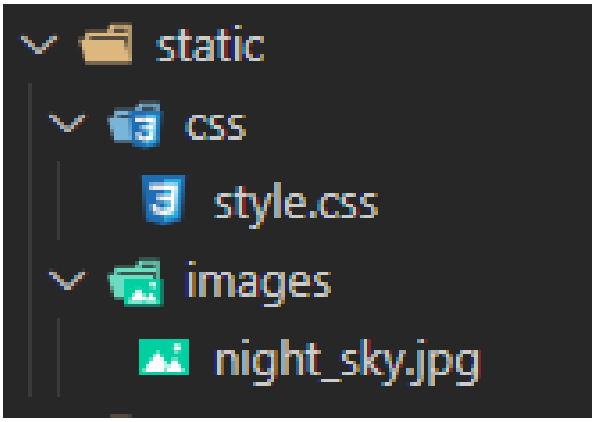


Рисунок 3 - Иерархия для статических файлов

В файле settings.py укажем каталог STATICFILES_DIRS.

STATICFILES_DIRS в Django — список дополнительных (нестандартных) путей к статическим файлам, используемых для сбора и для режима отладки (рисунок 4).

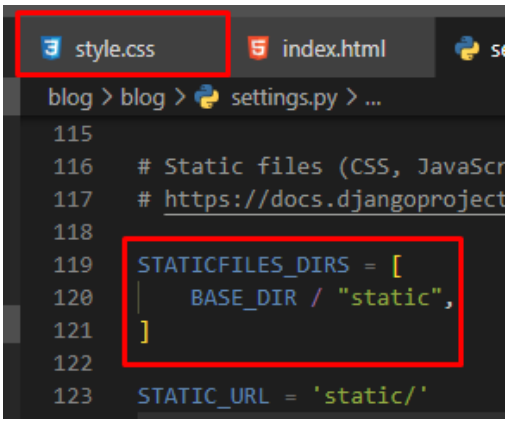


Рисунок 4 - Константа STATICFILES_DIRS

Создадим также папку templates и в ней файл index.html (рисунок 5).

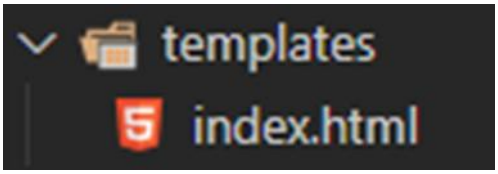


Рисунок 5 - Папка templates

Пропишем стили для сайта в файле стилей (рисунок 6).



Рисунок 6 - Стили для страницы index.html

Подключим файл стилей к index.html, а также разместим в теге первого

уровня текст «Ночное небо» и в теге изображений добавим путь к нашему изображению, используя все тот же язык DTL (Django Template Language). В качестве альтернативного текста (если картинки по каким-либо причинам не отобразится на страничке), установим все то же значение «Ночное небо» (рисунок 7).

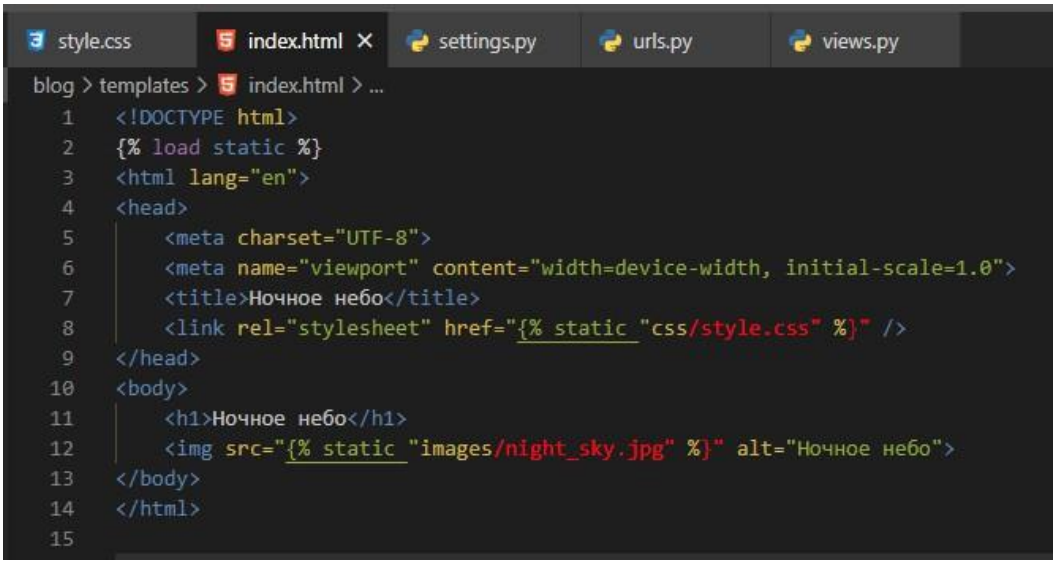


Рисунок 7 - Код для index.html

Результат нашей работы (рисунок 8):

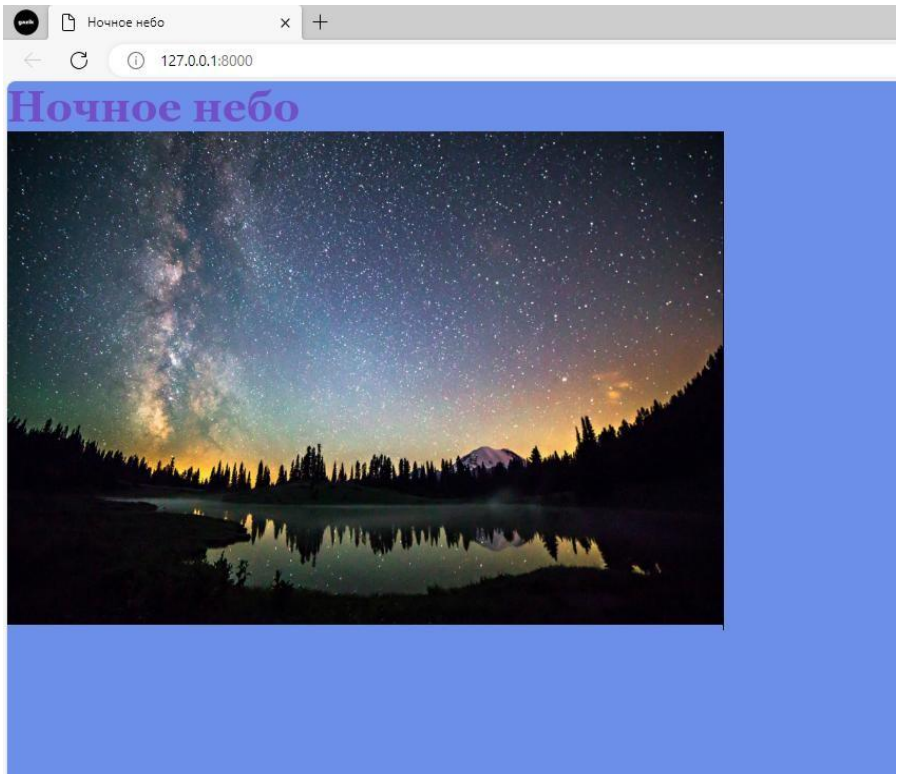


Рисунок 8 - Ночное небо с картинкой

4.2 Наследование шаблонов

Управляющими элементами шаблонов Django являются переменные, фильтры и теги.

При рендеринге шаблона переменные заменяются на свое значение, вычисленное в контексте вызова.

Синтаксис — двойные фигурные скобки, например: **{{ title }}.**

Фильтры служат для простых преобразований переменных.

Синтаксис — **переменная|имя_фильтра:"параметры"**. Фильтр может встречаться как в переменных, так и в качестве параметра тега.

Например: **{{ title|lowercase }}.**

При рендеринге шаблона теги, грубо говоря, заменяются на результаты работы ассоциированной с этим тегом функции на питоне.

Синтаксис: **{% тег параметры %}**

например: **{% url blog_article slug=article.slug %}**

Программист может написать свои фильтры и теги, но об этом позже. Кроме того, есть три специальных тега: include, block и extend.

Тег include подставляет запрошенный шаблон (отрендеренный в текущем контексте).

Все вышеперечисленное тривиально и в той или иной форме есть в любом движке шаблонов. Теперь перейдем к особенностям Django: на тегах block и extend строится наследование шаблонов. Остановимся на них подробнее.

Основная фишка шаблонов Django — наследование. Шаблон может расширять (уточнять) поведение родительского шаблона.

Любой участок шаблона может быть обернут в блочный тег (естественно, что тег не может начинаться перед, а заканчиваться внутри цикла). Блоку дается имя. Например, блок контент (рисунок 9):

```
1   {% block content %}
2       <!-- Содержимое страницы -->
3   {% endblock content %}
```

Рисунок 9 - Пример использования блоков

При помощи тега extend мы указываем, какой шаблон мы будем уточнять. Расширяя шаблон, мы можем переопределить любые блоки, которые есть в родительском шаблоне. Все, что находится вне этих блоков, будет пропущено.

Получается мощный механизм, практически исключающий необходимость повторения частей шаблонов. Давайте разберем реальный пример.

Предположим, мы разрабатываем веб-сайт, включающий в себя как обычные страницы, так и блог. Верстальщик предоставил нам макет одной страницы сайта, содержащий:

– шапку (логотип, меню);
– тело страницы;

– и подвал с информацией о правах распространения.

В качестве основы для дальнейшей работы мы создадим шаблоны, используя исключительно HTML и CSS, это позволит нам сфокусироваться на структуре и стилизации, отложив использование других технологий на более поздний этап.

Так выглядит index.html (рисунок 10):

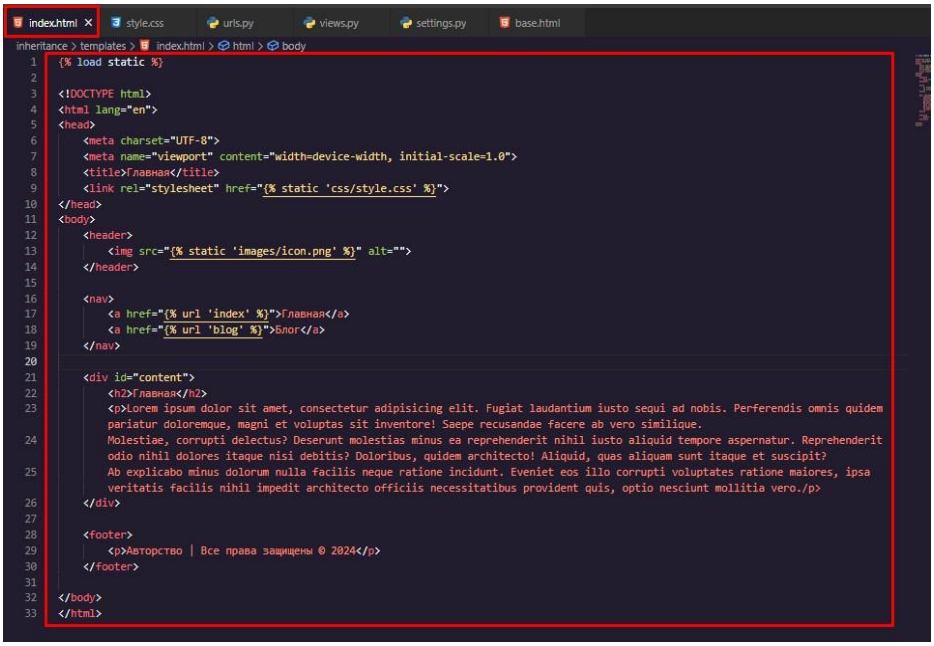


Рисунок 10 - Файл index.html

Так выглядит style.css (рисунок 11):



Рисунок 11 - Файл стилей (style.css)

Теперь, создадим страницу блога (рисунок 12):

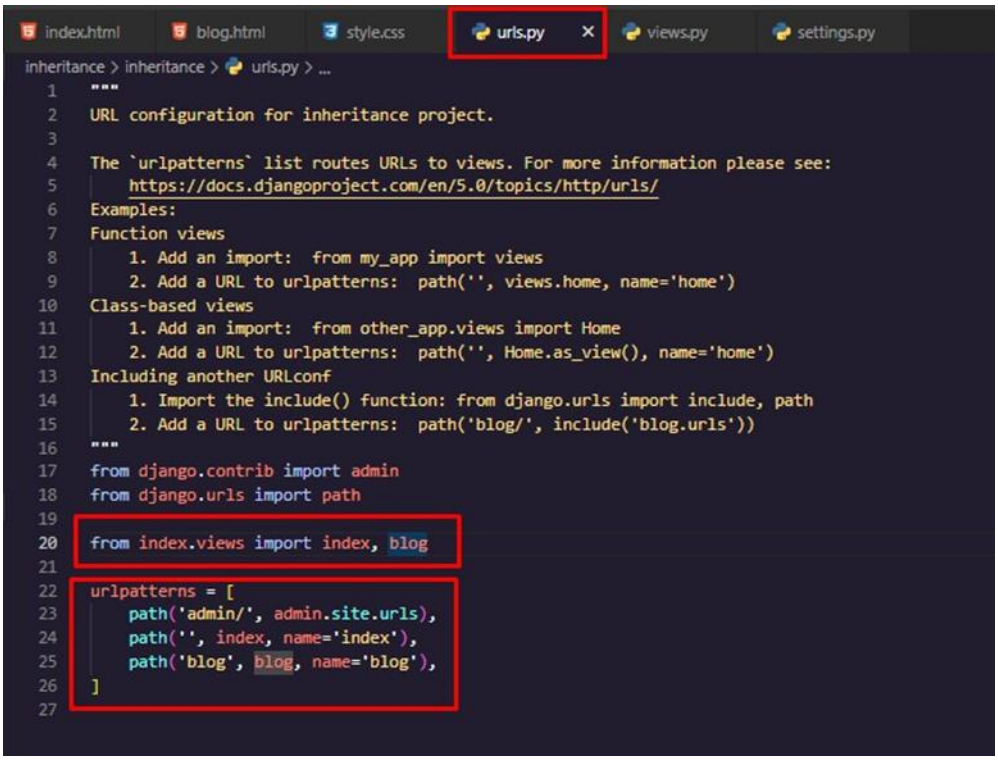


Рисунок 12 - Файл blog.html

Настроим url-шаблоны, для правильного перехода между страницами (рисунок 13).

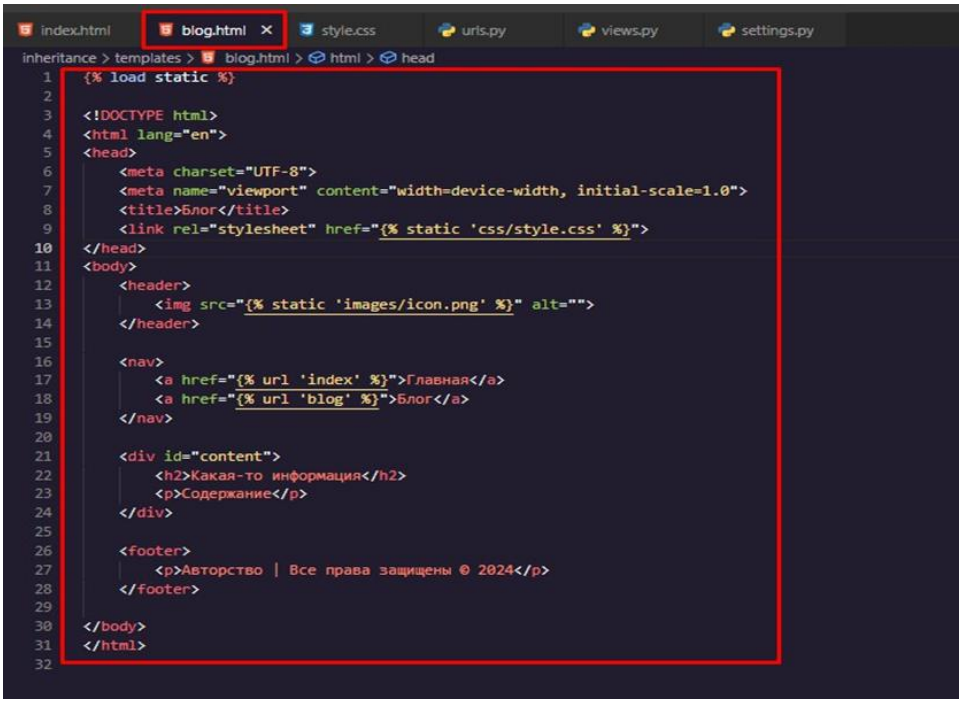


Рисунок 13 - Файл urls.py

Настроим отображение страниц в файле представлений (views.py) (рисунок

14).

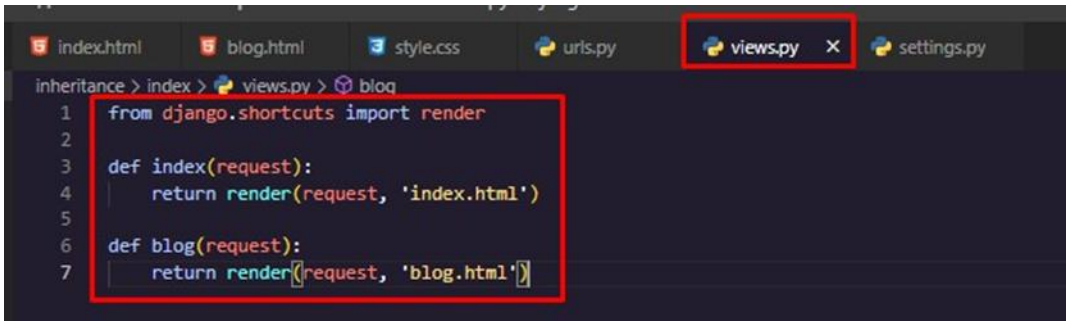


Рисунок 14 - Файл views.py

И в конечном итоге мы получаем (рисунки 15-16):

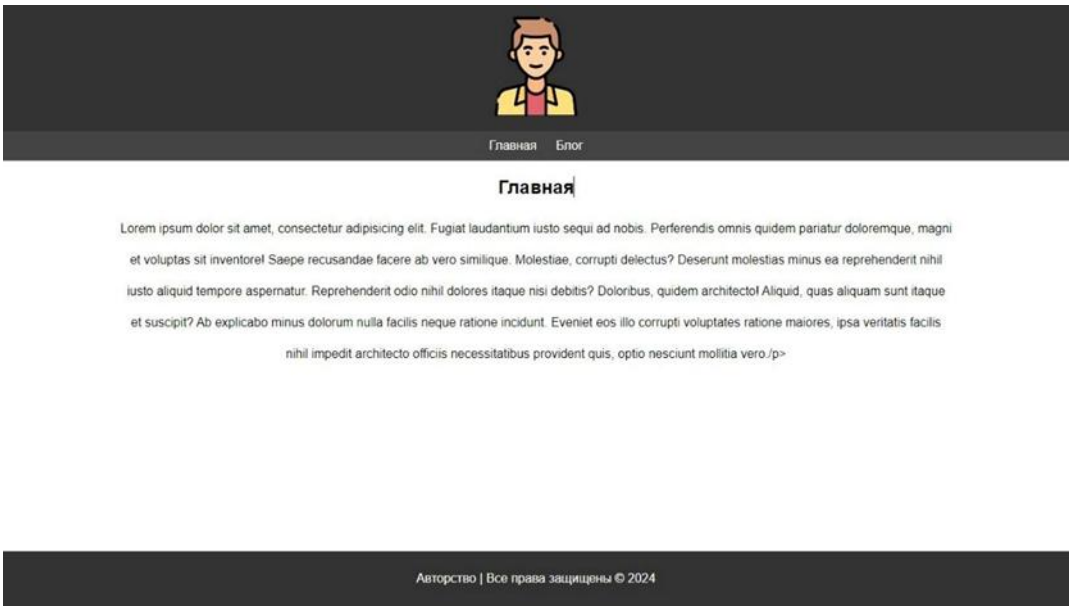


Рисунок 15 - Итоговый вид главной страницы

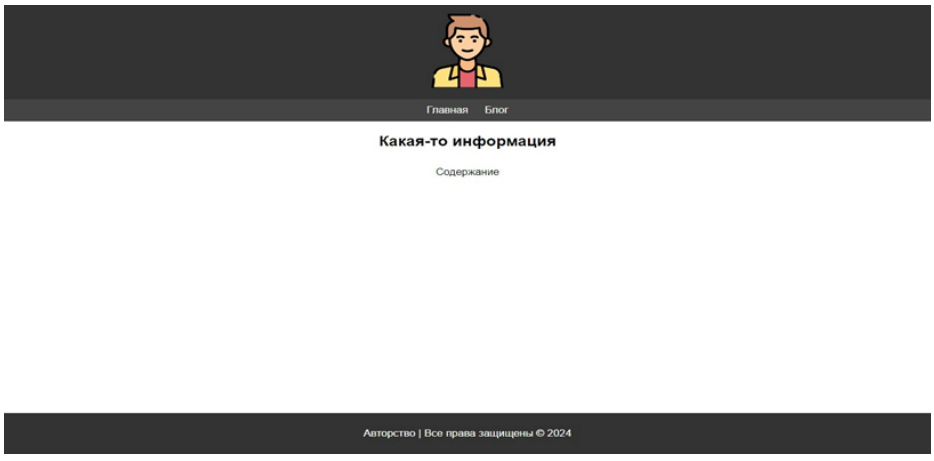


Рисунок 16 - Итоговый вид страницы blog.html

Если Вы заметили, то код index.html практически идентичен коду blog.html.

Создадим базовый шаблон под названием base.html и поместим, согласно теоретической части практической работы, код используя блоки. Добавим блок content, дабы в будущем мы могли расширять данный html-файл в других html-файлах (рисунок 17).

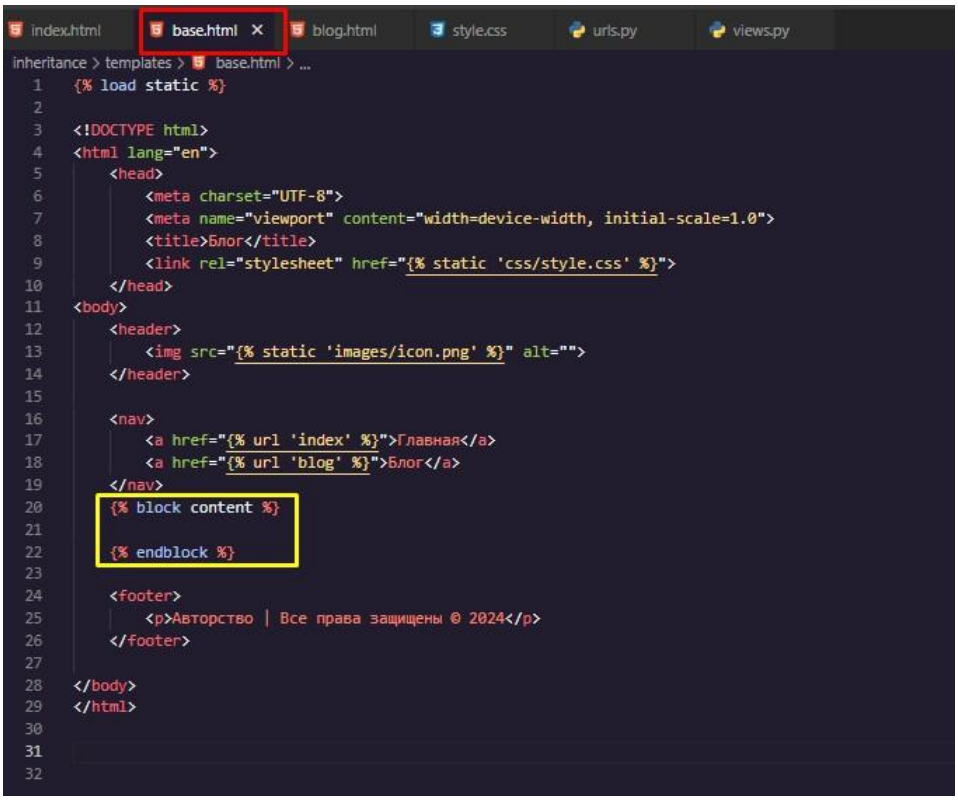


Рисунок 17 - Базовый шаблон base.html

Изменим код index.html на следующий (рисунок 18):



Рисунок 18 - Измененный index.html

Изменим код about.html на следующий (рисунок 19):

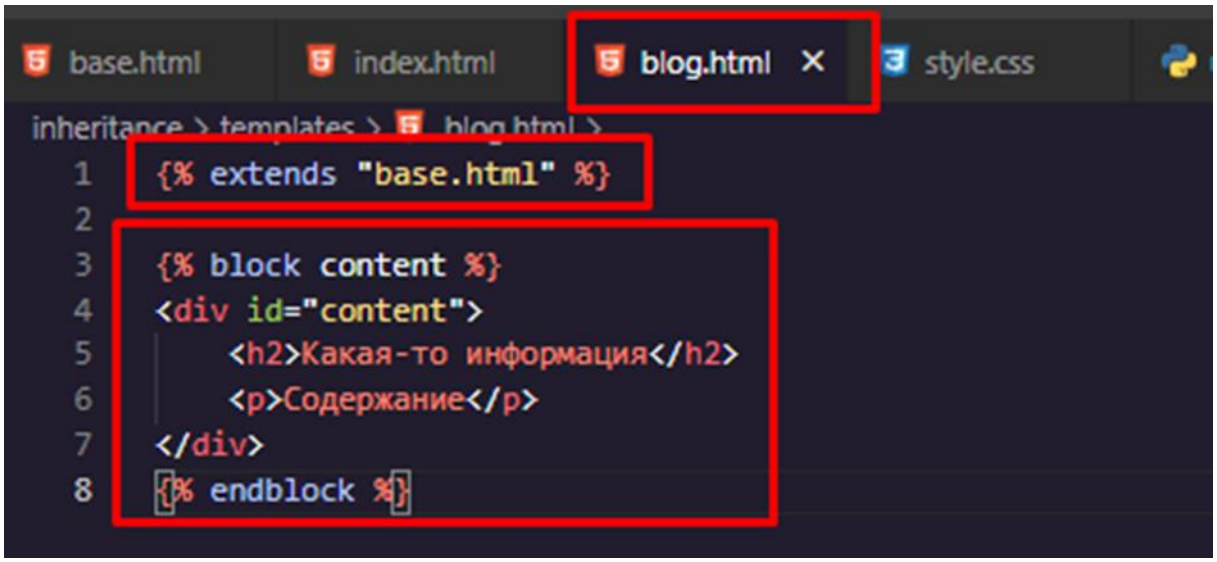


Рисунок 19 - Измененный blog.html

Перейдем на веб-сайт и посмотрим, что же изменилось (рисунки 20-21).



Рисунок 20 - Конечный результат страницы index.html

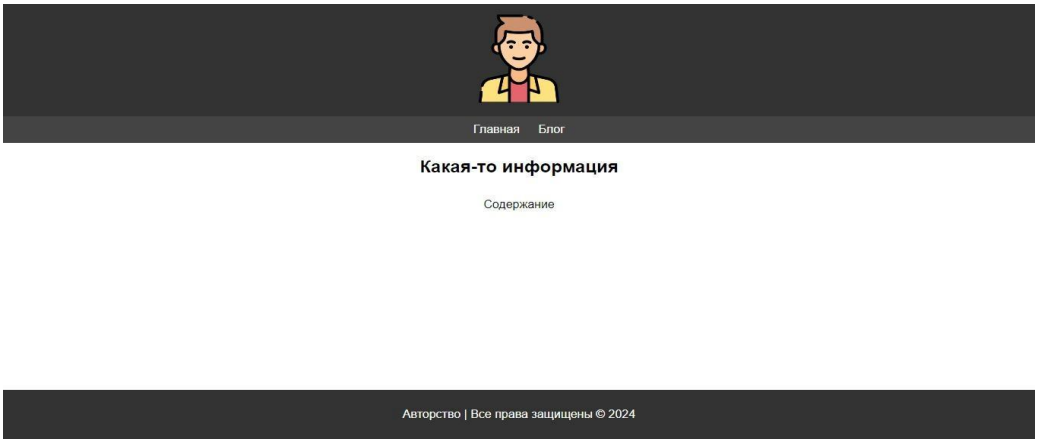


Рисунок 21 - Конечный результат страницы blog.html

Ничего не поменялось, потому что и не должно было меняться, ведь мы отобразили на сайте ту же самую структуру, но более правильно, не дублируя код. При надобности, мы можем добавить ещё блоки и использовать их как мы хотим.

4.3 Передача данных в шаблоны

Одним из преимуществ шаблонов является то, что мы можем передать в них динамически из представлений различные данные. Для вывода данных в шаблоне могут использоваться различные способы. Для вывода самых простых данных применяется двойная пара фигурных скобок:

{{ название_объекта }}

Например, пусть в проекте у нас есть папка templates, в которой содержится шаблон index.html:

Определим в файле index.html следующий код:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Django на METANIT.COM</title>
</head>
<body>
    <h2>{{ header }}</h2>
    <p>{{ message }}</p>
</body>
</html>
```

Здесь используется две переменных: message и header. Они будут передаваться из представления.

Чтобы из функции-представления передать данные в шаблон применяется третий параметр функции render, который еще называется context и который представляет словарь. Например, изменим файл views.py следующим образом:

```
from django.shortcuts import render
def index(request):
    data = {"header": "Hello Django", "message": "Welcome to Python"}
    return render(request, "index.html", context=data)
```

В шаблоне используются две переменных, соответственно словарь, который передается в функцию render через параметр context, теперь содержит два значения с ключами header и message.

В результате при обращении к корню веб-приложения мы увидим следующий вывод в браузере (рисунок 22):

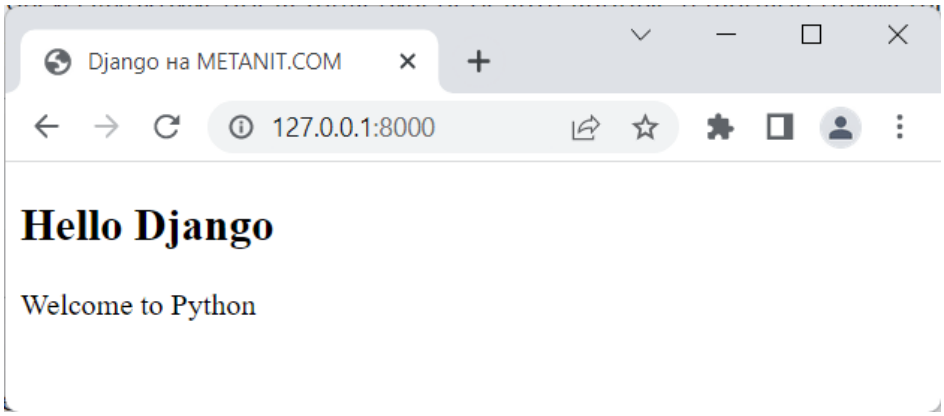


Рисунок 22 - Передача данных в шаблоны Templates в веб-приложении на Django

Рассмотрим передачу более сложных данных. Допустим, в представлении передаются следующие данные:

```
from django.shortcuts import render
def index(request):
    header = "Данные пользователя"            # обычная переменная
    langs = ["Python", "Java", "C#"]          # список
    user ={"name" : "Tom", "age" : 23}        # словарь
    address = ("Абрикосовая", 23, 45)         # кортеж
    data = {"header": header, "langs": langs, "user": user, "address": address}
    return render(request, "index.html", context=data)
```

В качестве третьего параметра в функцию render нам надо передать словарь, поэтому все данные оборачиваются в словарь и в таком виде передаются в шаблон.

В этом случае шаблон мог бы выглядеть, например, следующим образом:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Django на METANIT.COM</title>
</head>
<body>
    <h1>{{ header }}</h1>
    <p>Имя: {{ user.name}} Возраст: {{user.age}}</p>
    <p>Адресс: ул. {{address.0}}, д. {{address.1}}, кв. {{address.2}}</p>
    <p>Языки: {{langs.0}}, {{langs.1}}</p>
</body>
</html>
```

Поскольку объекты langs и address представляют соответственно массив и кортеж, то мы можем обратиться к их элементам через индексы, как мы бы работали бы с ними в коде на Python, например, первый элемент кортежа address: address.0.

Подобным образом, поскольку объект user представляет словарь, то мы можем обратиться к его элементам по ключам name и age: {{ user.name}} {{user.age}}.

В итоге мы получим следующий вывод в веб-браузере (рисунок 23):

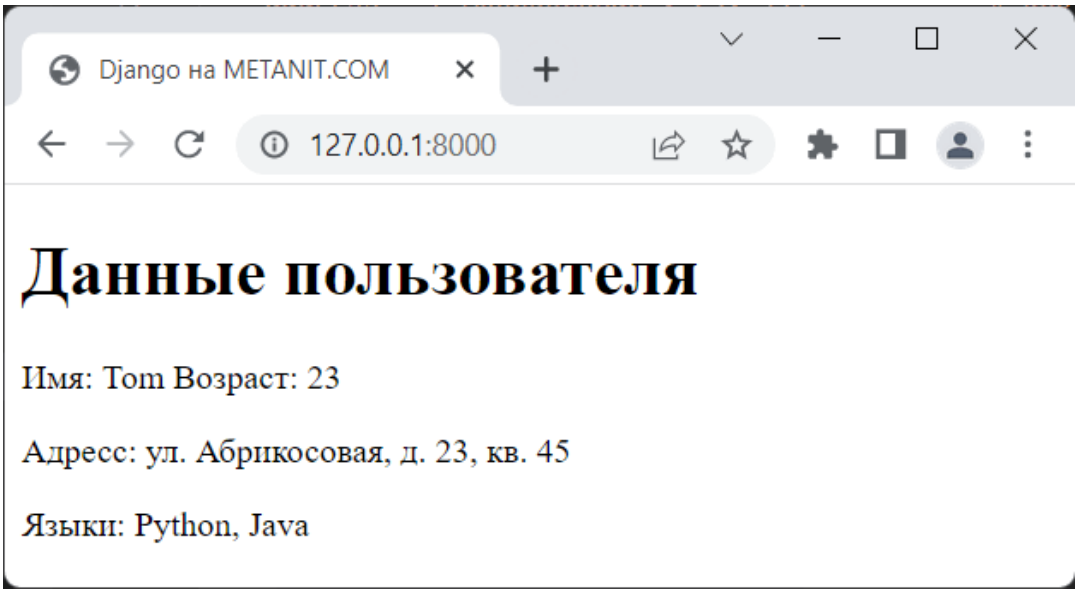


Рисунок 23 - Передача списков, кортежей и словарей в шаблон template в Django

Если для генерации шаблона применяется класс TemplateResponse, то в его конструктор также через третий параметр можно передать данные для шаблона:

```
from django.template.response import TemplateResponse
def index(request):
    header = "Данные пользователя"            # обычная переменная
    langs = ["Python", "Java", "C#"]          # список
    user ={"name" : "Tom", "age" : 23}        # словарь
    address = ("Абрикосовая", 23, 45)         # кортеж
    data = {"header": header, "langs": langs, "user": user, "address": address}
    return TemplateResponse(request,  "index.html", data)
```

Подобным образом можно передавать в шаблоны объекты своих классов. Например, определение функции-представления (рисунок 24):

```
from django.shortcuts import render
def index(request):
    return render(request, "index.html", context = {"person": Person("Tom")})
class Person:
    def __init__(self, name):
        self.name = name    # имя человека
```

Здесь определяется класс Person, в конструкторе которого передается некоторое значение для атрибута name. В функции index в шаблон передается объект с ключом "person".

В шаблоне index.html мы можем обращаться к функциональности объекта, например, к его атрибуту name:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Django на METANIT.COM</title>
</head>
<body>
    <h1>Person {{ person.name }}</h1>
</body>
</html>
```

Передача объектов классов в шаблон template в Django.

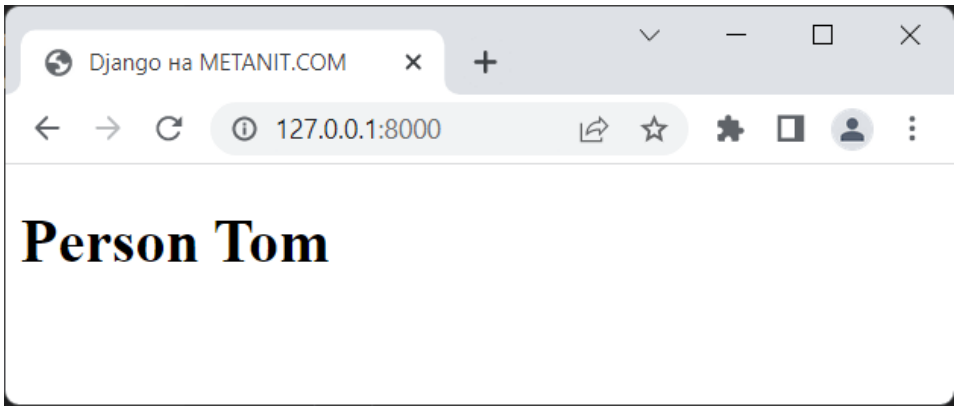


Рисунок 24 - Передача объектов классов в шаблон template в Django

4.4 Статические файлы

Веб-приложение, как правило, использует различные статические файлы - изображения, файлы стилей css, скриптов javascript и так далее. Рассмотрим, как мы можем использовать подобные файлы.

При создании проекта Django он уже имеет некоторую базовую настройку для работы со статическими файлами. В частности, в файле settings.py определена переменная STATIC_URL, которая хранит путь к каталогу со статическими файлами:

```
STATIC_URL = 'static/'
```

А среди установленных приложений в переменной INSTALLED_APPS указано приложение django.contrib.staticfiles (рисунок 25):

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'hello',
]
```

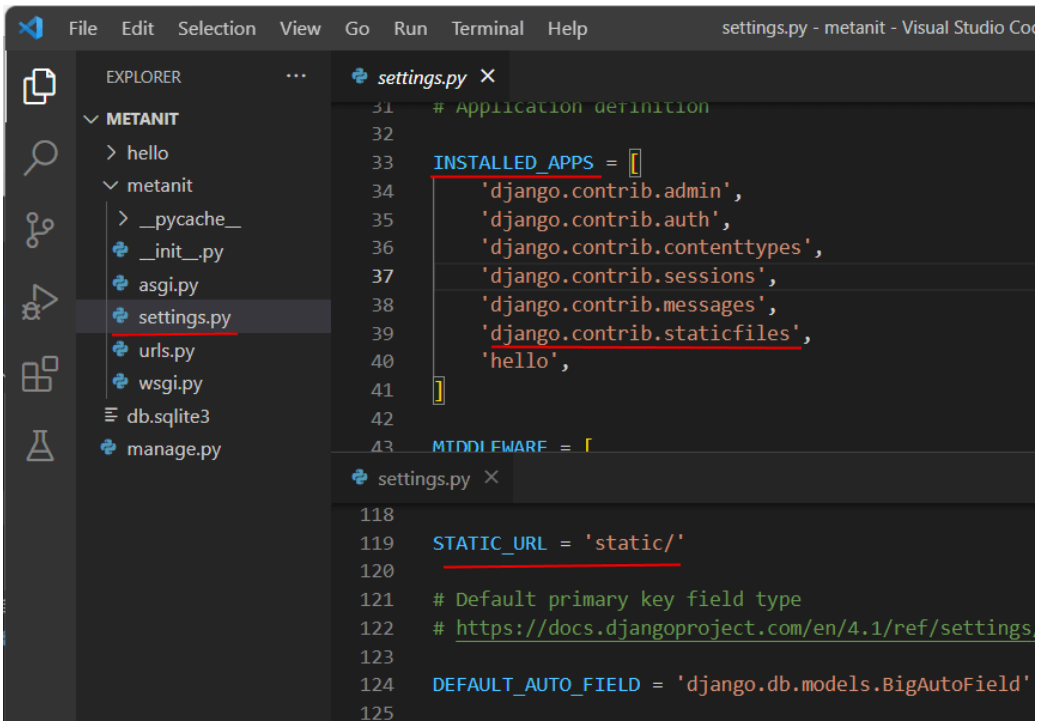


Рисунок 25 - Настройка статических файлов в файле settings.py в проекте на Django

Переменная STATIC_URL имеет значение "static/", а это значит, что нам достаточно создать в папке приложения каталог с именем "static" и добавить в него необходимые нам статические файлы. Но, естественно, при необходимости через данную настройку мы можем изменить расположение каталога статических файлов.

Итак, добавим в папку приложения новый каталог static. Чтобы не сваливать все статические файлы в кучу, определим для каждого типа файлов отдельные папки. В частности, создадим в папке static для изображений каталог images, а для стилей - каталог css. Подобным образом можно создавать папки и для других типов файлов.

В папку static/images добавим какое-нибудь изображение - в моем случае это будет файл forest.jpg. А в папке static/css определим новый файл styles.css, который будет иметь какие-нибудь простейшие стили, например (рисунок 26):

```
body{ font-family: Verdana;}
h1{color:navy;}
img{width:350px;}
```

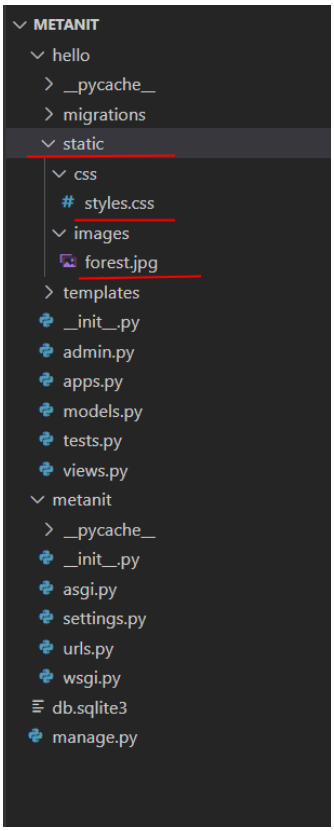


Рисунок 26 - Статические файлы в проекте на Django

Теперь используем эти файлы в шаблоне. Для этого в начале файла шаблона необходимо определить инструкцию

{% load static %}

При этом данный код должен идти после тега DOCTYPE.

Для определения пути к статическим файлам используются выражения типа

{% static "путь к файлу внутри папки static" %}

Так, пусть в приложении в папке templates определен шаблон index.html, который имеет следующий код (рисунок 27):

```
<!DOCTYPE html>
{% load static %}
<html>
<head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="{% static "css/styles.css" %}" />
    <title>Django на METANIT.COM</title>
</head>
<body>
    <h1>Зимний лес</h1>
    <img src="{% static "images/forest.jpg" %}" alt="зимний лес" >
</body>
</html>
```

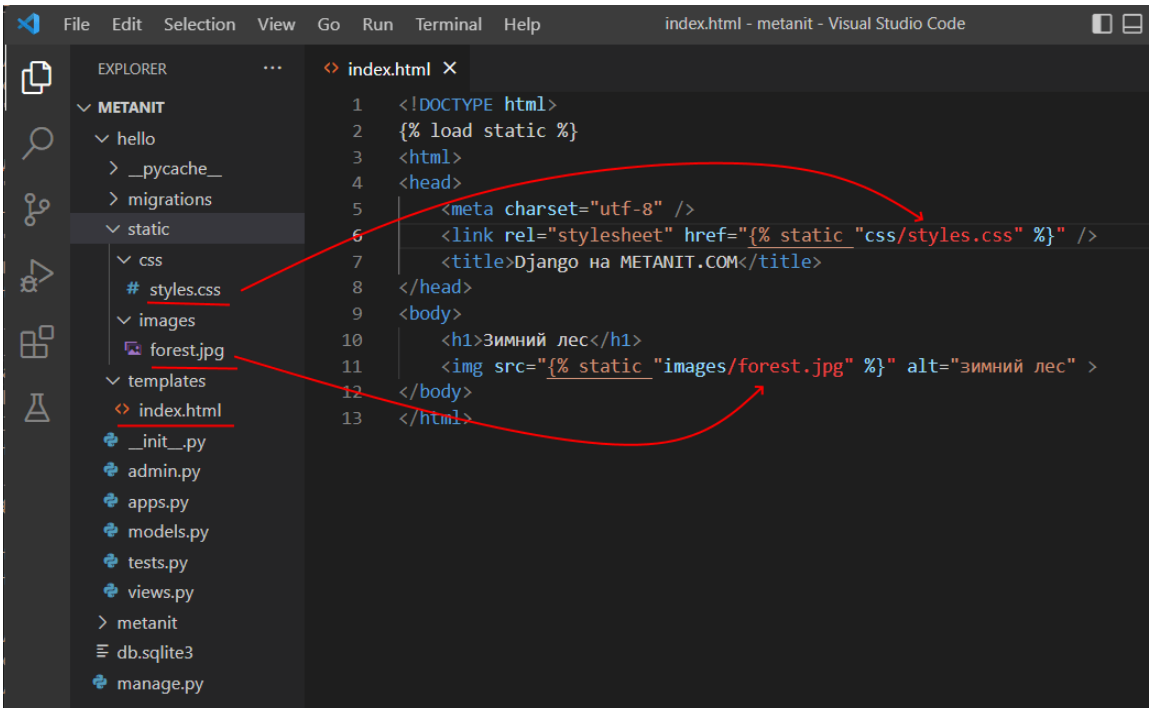


Рисунок 27 - Работа со статическими файлами в веб-приложении на Django

При запуске приложения шаблон index.html будет генерироваться в следующую веб-страницу, которая будет использовать изображение и применять стили (рисунок 28):

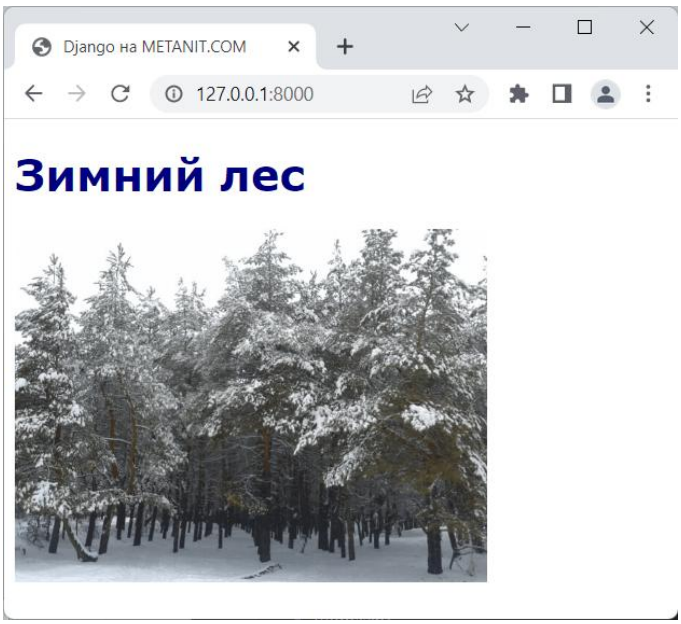


Рисунок 28 - Подключение статических файлов в приложении на Django и Python

Если нас не устраивает хранение файлов в каталоге по умолчанию - каталоге static, либо мы хотим указать несколько папок, то мы можем в файле settings.py задать все необходимые каталоги с помощью переменной STATICFILES_DIRS, которая принимает список путей:

```
STATICFILES_DIRS = [
   BASE_DIR / "static",
   "/var/www/static/",
   "/somefolder/"
]
```

4.5 Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте blog_site и приложении posts создайте папку static/posts/css. В этой папке создайте файл стилей style.css. Подключите созданный файл стилей в шаблоне base.html с помощью тегов {% load static %} и {% static %}. Убедитесь, что шаблон post_list.html наследуется от base.html с помощью тега {% extends %} и использует блок {% block content %}. Добавьте в файл CSS стили для оформления заголовка сайта, меню навигации и списка публикаций блога. Проверьте корректность отображения страницы в браузере.

Решение задачи:

1 Создание структуры каталогов

```
posts/
├── static/
│   └── posts/
│       └── css/
│           └── style.css
├── templates/
│   └── posts/
│       ├── base.html
│       └── post_list.html
```

2 Файл стилей (static/posts/css/style.css)

```
body {
   font-family: Arial, sans-serif;
   margin: 20px;
}
h1 {
   color: navy;
}
nav {
   margin-bottom: 20px;
}
nav a {
   text-decoration: none;
   margin-right: 15px;
   color: darkgreen;
}
ul {
   list-style-type: none;
   padding: 0;
```

```
}
li {
   padding: 10px;
   border-bottom: 1px solid #cccccc;
}
```

3 Базовый шаблон (templates/posts/base.html)

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
   <meta charset="UTF-8">
   <title>Блог</title>
   <link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
</head>
<body>
<h1>Блог публикаций</h1>
<nav>
   <a href="/">Главная</a>
</nav>
{% block content %}
{% endblock %}
</body>
</html>
```

4 Шаблон списка публикаций (templates/posts/post_list.html)

```
{% extends 'posts/base.html' %}
{% block content %}
<h2>Список публикаций</h2>
<ul>
   {% for post in posts %}
      <li>
         <strong>{{ post.title }}</strong><br>
         Автор: {{ post.author }}
      </li>
   {% empty %}
      <li>Публикации отсутствуют.</li>
   {% endfor %}
</ul>
{% endblock %}
```

5 Представление (views.py)

```
from django.shortcuts import render
from .models import Post
def post_list(request):
   posts = Post.objects.all()
   return render(request, 'posts/post_list.html', {'posts': posts})
```

6 Маршруты (posts/urls.py)

```
from django.urls import path
from .views import post_list
urlpatterns = [
    path('', post_list, name='post_list'),
]
```

7 Запуск проекта

```
python manage.py runserver
```

После открытия адреса http://127.0.0.1:8000/ должна отображаться страница со списком публикаций блога, оформленная с использованием подключённого файла стилей style.css.

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 В ранее созданном проекте library_site и приложении books создайте папку static/books/css. В этой папке создайте файл стилей style.css. Затем измените шаблон base.html, подключив созданный файл стилей с помощью тегов {% load static %} и {% static %}. Убедитесь, что шаблон book_list.html наследуется от base.html с помощью тега {% extends %} и использует блок {% block content %}. Добавьте в файл CSS стили для заголовка сайта и меню навигации.

2 В ранее созданном проекте student_portal и приложении students создайте папку static/students/css. В этой папке создайте файл стилей style.css. Затем измените шаблон base.html, подключив файл стилей с помощью тегов {% load static %} и {% static %}. Убедитесь, что шаблон student_list.html наследуется от base.html с помощью {% extends %} и использует блок {% block content %}. Добавьте в файл CSS стили для оформления списка студентов.

3 В ранее созданном проекте movie_portal и приложении films создайте папку static/films/css. В этой папке создайте файл style.css. Подключите данный файл в шаблоне base.html с помощью {% load static %} и {% static %}. Убедитесь, что шаблон film_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка фильмов.

4 В ранее созданном проекте music_library и приложении albums создайте папку static/albums/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон album_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка музыкальных альбомов.

5 В ранее созданном проекте online_shop и приложении products создайте папку static/products/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html с помощью {% static %}. Убедитесь, что шаблон product_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления таблицы товаров интернет-магазина.

6 В ранее созданном проекте news_portal и приложении news создайте папку static/news/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон news_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка новостей.

7 В ранее созданном проекте tourism_portal и приложении tours создайте папку static/tours/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон tour_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка туристических маршрутов.

8 В ранее созданном проекте sport_club_site и приложении teams создайте папку static/teams/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон team_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка спортивных команд.

9 В ранее созданном проекте school_portal и приложении teachers создайте папку static/teachers/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон teacher_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка преподавателей.

10 В ранее созданном проекте hospital_system и приложении patients создайте папку static/patients/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон patient_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка пациентов.

11 В ранее созданном проекте education_platform и приложении courses создайте папку static/courses/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон course_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка учебных курсов.

12 В ранее созданном проекте restaurant_site и приложении menu создайте папку static/menu/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон dish_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка блюд ресторана.

13 В ранее созданном проекте car_catalog и приложении cars создайте папку static/cars/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html. Убедитесь, что шаблон car_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка автомобилей.

14 В ранее созданном проекте portfolio_site и приложении main создайте папку static/main/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html с помощью тегов {% load static %} и {% static %}. Убедитесь, что шаблон project_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка проектов портфолио.

15 В ранее созданном проекте home_site и приложении home создайте папку static/home/css. В этой папке создайте файл style.css. Подключите файл стилей в шаблоне base.html с помощью тегов {% load static %} и {% static %}. Убедитесь, что шаблон page_list.html наследуется от base.html и использует блок {% block content %}. Добавьте стили для оформления списка страниц сайта.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы
6.2 Цель лабораторной работы
6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Укажите, где располагаются статические файлы в проекте Django в языке программирования Python.
7.2 Объясните, для чего используется папка static в языке программирования Python.
7.3 Опишите, как подключить статические файлы в шаблоне Django в языке программирования Python.
7.4 Объясните, для чего используется тег {% load static %} в языке программирования Python.
7.5 Перечислите, какие элементы обычно содержит файл base.html в языке программирования Python.
7.6 Охарактеризуйте, что такое наследование шаблонов в языке программирования Python.
7.7 Объясните, для чего используется тег {% extends %} в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.
2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.
3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.